SkillConnect: Complete Code & Syntax Guide

This guide is designed for beginners and college project presentations (viva / evaluation). It explains what the code does, what each syntax keyword means, and how the entire system connects together.

1. The Core Architecture: Django MVT Pattern

SkillConnect follows the **Model - View - Template (MVT)** design pattern:

```
text
Browser Request
      ↓
  urls.py       (Route finder: "Where should this request go?")
      ↓
  views.py      (The Brain / Logic: "What calculations & data are needed?")
      ↓
  models.py     (The Database Blueprint: "How is data structured in MySQL?")
      ↓
templates/*.html (The Presentation: "How should the final page look?")
      ↓
Browser Response (HTML + CSS + JavaScript)
```

2. Models (`models.py`) — The MySQL Database Blueprint

What is a Model?

In Django, you do **not** write raw SQL like `CREATE TABLE`. Instead, you write Python classes. Django reads these classes and automatically generates the matching SQL tables inside MySQL (`skillconnect_db`).

Real Code Example:

```
python
class StudentProfile(models.Model):
    # 1. Foreign Key: Links this student profile to a specific User
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='student_profile')

    # 2. Text fields with maximum character length constraints
    college_name = models.CharField(max_length=200)
    course = models.CharField(max_length=150)
    year_of_study = models.CharField(max_length=50)

    # 3. File upload field: saves image file to 'media/college_ids/'
    college_id_card = models.ImageField(upload_to='college_ids/', blank=True, null=True)

    # 4. Choice field with predefined dropdown options
    VERIFICATION_STATUS_CHOICES = (
        ('PENDING', 'Pending Verification'),
        ('VERIFIED', 'Verified Student'),
        ('REJECTED', 'Rejected'),
    )
    verification_status = models.CharField(max_length=20,
choices=VERIFICATION_STATUS_CHOICES, default='PENDING')

    # 5. Many-to-Many Relationship: One student can have multiple skills
    skills = models.ManyToManyField(Skill, blank=True, related_name='student_profiles')
```

Syntax Breakdown:

- `class StudentProfile(models.Model)`: Creates a new table named `accounts_studentprofile` in MySQL.
- `OneToOneField(User, on_delete=models.CASCADE)`:
  - **OneToOneField**: Guarantees that 1 user has exactly 1 student profile.
  - **on_delete=models.CASCADE**: If the user is deleted, their student profile is automatically deleted too (no orphaned rows).
- `CharField(max_length=200)`: Translates to MySQL `VARCHAR(200)`.
- `ImageField(upload_to='college_ids/')`: Stores the filename string in MySQL (e.g. `college_ids/id.jpg`) while saving the physical file to the hard drive inside the `media/college_ids/` folder.

- `ManyToManyField(Skill)` : Creates an intermediate junction table in MySQL linking students to skills.
- `blank=True, null=True` : Means this field is optional in forms (`blank=True`) and can be `NULL` in the database (`null=True`).

3. Views (`views.py`) — The Brain & Business Logic

What is a View?

A view is a Python function that:

1. Receives a web request (`request`).
2. Checks user permissions (e.g. "Is this user logged in?", "Is this an admin?").
3. Queries or modifies MySQL through models.
4. Renders the template with data or returns a JSON response.

Real Code Example:

```
python
@login_required
def book_service_view(request, service_id):
    # 1. Fetch the service or return 404 Not Found if it does not exist
    service = get_object_or_404(Service, id=service_id, is_active=True)

    # 2. Validation check: Prevent student from hiring themselves
    if service.student == request.user:
        messages.warning(request, "You cannot book your own service.")
        return redirect('service_detail', service_id=service.id)

    # 3. Check HTTP method: POST = user submitted form; GET = user just opened page
    if request.method == 'POST':
        form = OrderBookingForm(request.POST)
        if form.is_valid():
            # 4. Create object in memory before saving to set relations
            order = form.save(commit=False)
            order.customer = request.user
            order.student = service.student
            order.service = service
            order.price = service.price
            order.status = 'REQUESTED'
            order.save() # Saves into MySQL!

            # 5. Dispatch notification to the student
            Notification.send(
                recipient=order.student,
                sender=request.user,
                notification_type='ORDER_REQUEST',
                title=f'New Order: {service.title}',
                message=f"{request.user.get_display_name()} requested your service.",
                link=f"/orders/{order.id}/"
            )

            # 6. Redirect to visual order tracking page
            return redirect('order_detail', order_id=order.id)
    else:
        form = OrderBookingForm()

    # 7. Render HTML template with context variables
    return render(request, 'orders/book_service.html', {'service': service, 'form': form})
```

Syntax Breakdown:

- `@login_required`: A **Python Decorator**. If an unauthenticated visitor tries to open this page, Django automatically redirects them to `/login/`.
- `request.method == 'POST'`: Distinguishes between viewing a page (`GET`) and submitting data (`POST`).
- `commit=False`: Tells Django: "*Create the Python object from the form, but wait before writing to MySQL until I add* `customer` *and* `student`."
- `form.is_valid()`: Automatically checks security rules, required fields, and input formats.
- `messages.warning()`: Shows a flash alert message on the screen.
- `render(request, 'template.html', context_dict)`: Merges the HTML file with Python variables.

4. Templates (`templates/*.html`) — The Presentation Layer

What is Django Template Language (DTL)?

HTML is normally static. Django Template Language allows inserting dynamic variables, loops, and conditions into HTML.

Real Code Example:

```
html
<!-- 1. Template Inheritance: borrows header, navbar, footer from base.html -->
{% extends 'base.html' %}

{% block content %}
<div class="card">
  <!-- 2. Variable Display using double curly braces -->
  <h2>Order #{{ order.order_number }}</h2>
  <p>Price: ₹{{ order.price }}</p>

  <!-- 3. Conditional rendering: checks status and user role -->
  {% if order.status == 'REQUESTED' %}
    <span class="badge badge-pending">Requested</span>

    <!-- Only the student sees the Accept/Reject buttons -->
    {% if is_student %}
      <form action="{% url 'update_order_status' order.id %}" method="POST">
        <!-- 4. Cross-Site Request Forgery security token -->
        {% csrf_token %}
        <input type="hidden" name="status" value="ACCEPTED">
        <button type="submit" class="btn btn-success">✓ Accept Order</button>
      </form>
    {% endif %}

  {% elif order.status == 'COMPLETED' %}
    <span class="badge badge-verified">✓ Completed</span>
  {% endif %}

  <!-- 5. For-loop over database query results -->
  <div class="timeline">
    {% for history in order.status_history.all %}
      <div>
        <strong>{{ history.get_status_display }}</strong> at {{ history.created_at|date:"M
d, H:i" }}
      </div>
    {% endfor %}
  </div>
</div>
{% endblock %}
```

Syntax Breakdown:

- `{{ variable.property }}`: Evaluates and prints data (e.g. `{{ order.price }}`).

- `{% if condition %} ... {% elif %} ... {% endif %}`: Shows or hides elements based on logic.
- `{% for item in list %} ... {% endfor %}`: Repeats an HTML block for each item in a database list.
- `{% csrf_token %}`: Inserts a cryptographically signed hidden token to protect against malicious cross-site attacks.
- `{% url 'route_name' parameter %}`: Generates dynamic URLs without hardcoding `/orders/5/`.

5. Vanilla JavaScript (`static/js/*.js`) — The Client-Side Interactivity

What does the JavaScript do?

SkillConnect uses pure Vanilla JavaScript (no React/Node needed) for:

1. **AJAX Likes**: Likes a post without refreshing the whole webpage.
2. **AJAX Polling Chat**: Checks every 3 seconds for new messages.
3. **Share to Clipboard**: Copies the link and shows a toast banner.

Real Code Example: AJAX Like Button in

```
javascript
// 1. Listen for user click on any like button
document.querySelectorAll('.btn-like').forEach(button => {
  button.addEventListener('click', async (event) => {
    event.preventDefault(); // Stop normal browser page jump

    const postId = button.dataset.postId; // Get data-post-id attribute

    // 2. Send background asynchronous HTTP request to Django
    const response = await fetch(`/posts/${postId}/like/`, {
      method: 'POST',
      headers: {
        'X-CSRFToken': getCookie('csrftoken'), // Security token
        'X-Requested-With': 'XMLHttpRequest'
      }
    });

    // 3. Receive JSON response from Django: { "liked": true, "count": 5 }
    if (response.ok) {
      const data = await response.json();
      const countSpan = button.querySelector('.like-count');
      const icon = button.querySelector('.like-icon');

      // 4. Update DOM immediately without reloading
      countSpan.textContent = data.count;
      if (data.liked) {
        icon.textContent = '♥';
        button.classList.add('liked');
      } else {
        icon.textContent = '♡';
        button.classList.remove('liked');
      }
    }
  });
});
```

Syntax Breakdown:

- `document.querySelectorAll('.btn-like')`: Finds all like buttons on the page.
- `event.preventDefault()`: Prevents default browser behaviors (such as submitting or jumping).
- `fetch(url, options)`: Modern native JavaScript HTTP client replacing old XMLHttpRequest.

- `await response.json()`: Parses JSON data sent back from Django.
- `countSpan.textContent = data.count`: Updates the number live on the screen.

6. Real-Time Chat Polling (`static/js/chat.js`)

```
javascript
// Polls the Django backend every 3000 milliseconds (3 seconds)
setInterval(async () => {
  const response = await fetch(`/chat/${conversationId}/poll/?last_id=${lastMessageId}`);
  if (response.ok) {
    const data = await response.json();
    if (data.messages && data.messages.length > 0) {
      data.messages.forEach(msg => {
        appendMessage(msg); // Appends new bubble to chat
        lastMessageId = Math.max(lastMessageId, msg.id);
      });
    }
  }
}, 3000);
```

Why Polling?

Instead of requiring complicated external WebSocket servers or Node.js infrastructure, AJAX polling runs directly on standard Python Django and MySQL, making the code 100% reliable and easy to deploy and demonstrate.

7. Viva / Exam Questions & Answers

Question	Answer
Why MySQL over SQLite?	MySQL is a multi-user, ACID-compliant relational database that handles concurrent writes, real foreign keys, and indexes required for production marketplaces.
How are passwords stored?	Django uses PBKDF2 with SHA-256 password hashing and a cryptographic salt. Passwords are never stored as plain text.
How does student verification work?	Students upload college enrollment details and ID images. Admins review them via a dedicated console and toggle the verification state to `VERIFIED`, which triggers the badge and notifies the student.
How does the recommendation feed work?	It cross-references `Profile.interests` with `Post.category`, surfaces posts from creators the user follows, and boosts posts from Verified Students.
How is CSRF prevented?	Django generates a CSRF token per session. Every POST request and AJAX call validates this token (`X-CSRFToken`) before modifying database rows.